Proof-Carrying Neurosymbolic Supervision: State, Reasoning, and Evidence Reuse

Anonymous Author(s)
Affiliation
Address
email

Abstract

Coding agents repeatedly reconstruct repository context, reason about changes, and validate results. We study an obligation-centered supervision architecture that retains source-linked semantic state, routes supported obligations through checked symbolic mechanisms, and uses neural proposals for unresolved work. Evidence applicability governs validation reuse and accepted-state publication. The empirical question is whether their combined startup and operating costs reduce neural work and validation effort while preserving useful progress. We evaluate context substitution, avoided model calls, persistent planning, validation reuse and native proof consumption as distinct mechanisms, retaining an earlier additive-context cold-repair study as separate evidence. On four constructed tasks, symbolic repair and residual routing pass the same independent checks as a model-first baseline with 2,430 versus 4,865 provider tokens. This 50.05% reduction is confined to that development suite. Other experiments exercise lifecycle-preserving reuse, memory consumption, publication, crash recovery and proofs over actual saved state. Simple retrieval matches memory's action reduction, and measured reuse savings do not recover priming cost over five repetitions. In two development comparisons on one natural two-issue repository sequence, every arm accepts one issue and none completes the trajectory. These mixed findings establish bounded capabilities and implementation limitations. Complete integrated token savings remain open; component percentages cannot be combined into a system estimate.

1 Problem and contribution

The cost of a coding agent includes repeated repository discovery, specification interpretation, planning, repair, validation, and recovery. Persistent state can avoid some of this work only when the state remains applicable to the current program. A shorter prompt can omit necessary dependencies; a cached pass can refer to obsolete source. Repository-agent interfaces and test-validation benchmarks make this boundary between proposals and checks central to useful repair [13, 5].

Our proposed method has three connected contributions. First, source and intent produce identity-bound semantic facts and explicit unresolved obligations. Second, those obligations guide deterministic analysis, checked reasoning and bounded synthesis before an authorized neural residual. Third, validation, evidence reuse and accepted-state publication preserve the scope and applicability of each observation. Accepted changes invalidate affected facts and plans, providing the next iteration with persistent, current evidence.

The resulting hypothesis concerns the entire workflow: repeated neural reasoning and validation may be displaced by persistent, checked work. Context compression is one possible mechanism. Fewer model calls and less replanning are distinct mechanisms, while test reuse primarily reduces validation

Submitted to NeurIPS 2026 Workshop on AI for Verifiable Coding. Do not distribute.

computation. Each requires its own correctness and cost comparison. No benefit follows merely from having a symbolic component in the source tree.

We retain an earlier additive-context cold-repair study as one bounded experiment, then evaluate missing mechanisms incrementally. Architecture, local qualification, development pilots and confirmatory effectiveness have different evidential roles. A failed or inconclusive experiment remains a result; an unrun comparison remains an open research obligation. In particular, the original 32 cold cells cannot establish the value of persistent reasoning or whole-system token savings.

2 The supervision loop

Compile state and obligations. The semantic index extracts source-linked symbols, dependencies, contracts and analysis limits. Intent supplies candidate goals and permitted effects; formalization produces bounded obligations without turning an inferred specification into authority. Semantic state is separate from operational task state and artifact storage. Missing dependencies, dynamic features and lossy translations remain an explicit unknown frontier that requires more source, broader checking or abstention.

Reason before proposing. For each obligation, the intended supervisor considers applicable evidence or procedures, exact dependency and static analysis, contract/solver checks, and bounded deterministic repair or synthesis. A neural model proposes unresolved work through a permitted residual route. Abstract interpretation, translation validation and counterexample-guided methods provide established foundations; the contribution is their composition at the supervisor's decision boundary, not a new general-purpose prover [2, 1, 10, 12]. The evaluation must demonstrate that a computed result changes the next actual action and passes independent checks. A route label or a candidate's assertion that it closes a claim is insufficient.

Validate, reuse and publish. Candidate edits are checked against the current source and required acceptance conditions. Evidence reuse requires matching subjects, dependency and environment identities, declared checker semantics and trust policy. For fixture-based tests, collection and materialized setup state must be resolved before call-phase reuse; required setup and teardown effects still run, and teardown failure prevents whole-item success. A complete accepted-transition manifest binds the current parent and applicable evidence before publication. Restart reconciliation must support valid continuation as well as rejection of stale work.

Persist, invalidate and replan. An accepted transition updates the state used by the next task. Source, dependency, fixture, policy and capability changes invalidate affected evidence and reopen obligations. Procedural memory can nominate previously useful actions, but applicability and independent validation remain necessary. Learned world models and procedure discovery require held-out downstream evaluation; their source presence does not establish generalization. The intended savings appear over trajectories of tasks, so a single cold proposal is an incomplete test of this architecture.

Separate resource hypotheses. For a fixed workload of N tasks and arm a, let

$$T_a(N) = T_{\mathrm{setup},a} + \sum_{j \in \mathcal{C}_a(N)} T_j,$$

where $\mathcal{C}_a(N)$ contains every workflow model call, including planning, formalization, proposals, review, failed attempts, retries and recovery. Provider-reported totals are preserved without adding overlapping cached or reasoning-token fields. Unknown usage remains unknown. The reduction $1 - T_{\mathrm{system}}(N)/T_{\mathrm{baseline}}(N)$ is meaningful only for matched workloads with adequate independently measured progress and comparable complete accounting. Report setup and priming at explicit reuse horizons; provider charges, host computation and elapsed time are separate quantities. Independent experimental oracle work is reported separately from each arm's runtime validation, without hiding mandatory runtime costs.

3 Method and trusted assumptions

Separate evidence classes. Semantic state links facts, dependencies, contracts, and unresolved obligations to source. Operational state records goals, tasks, attempts, leases, and decisions. Artifact storage preserves candidate bytes and accepted manifests. An evidence item binds a subject to a declared statement, checker, environment, and provenance. A passing test concerns the finite checks actually executed; a formal derivation concerns its encoded relation and premises; a grant concerns an actor's permission to act now. These classes cannot substitute for one another. In particular, a signature binds a record to a key under cryptographic assumptions. It establishes neither checker adequacy nor the honesty of an observation.

Applicable evidence. The intended loop derives obligations from current source, chooses an admissible analysis or proposal route, checks an isolated candidate, and admits only evidence applicable to that candidate. Translation validation motivates checking a particular conversion against its stated relation [10]; a source-linked record alone is not such validation. Abstract facts likewise depend on an approximation relation [2]. A typed or content-addressed representation does not imply full source-language soundness.

Conditional acceptance and reuse. If publication succeeds only after checking the current parent, authorized scope, complete requirement manifest, admitted evidence, and expected-parent compare-and-swap, each successful successor satisfies those gates. Given a valid initial root and exclusive use of that publication operation, induction preserves the declared manifest policy. This does not establish enforcement by unexamined write paths or atomicity of external effects.

For test reuse, suppose the observation of test t under profile θ factors through a complete dependency projection:

$$\mathrm{Obs}_\theta(t, s) = f_\theta(\pi_D(s)).$$

If f_θ is deterministic on every admitted state, equal projections imply equal observations. Completeness and determinism are substantive premises; hashing an incomplete dependency list, opaque fixture, or uncontrolled effect does not discharge them. Appendix A preserves the full conditional, unmachine-checked statements and the distinction between behavioral acceptance, action permission, and publication.

Executed scope. The historical cold comparison exercises native context preparation, model proposals, candidate custody and AI inspection, and separate cold scoring. Earlier constructed controls and a local guard-before-write witness test named implementation boundaries; they do not establish that the complete intended loop ran as a live benchmark. Its original proof profile lacks the admitted backend, key provenance, and exact statement binding required for native proof admission. New development profiles and their measured results are reported separately in Section 6; neither native proving performance nor empirical reuse benefit is inferred from the conditional arguments.

4 Historical component-study design

The retained cold study compares raw selected public context (A) with the same raw selection plus native source-linked semantic context (B). Eight convenience-selected Python repository families and two nested repetitions per arm yield 32 fixed cells. Sixteen family/arm request commitments precede final generation. The treatment is additive and cannot isolate context compression. The retained A/B cells form an order-preserving subset of a later 192-cell plan; 160 factor cells were withdrawn and eight additional target families were not recruited. These missing comparisons remain separate research obligations.

Each cell permits one Grok 4.6 proposal, requesting at most 4,096 output tokens, with no model retry, scientific fallback or served random seed. The nominal gateway bound is 600 seconds; actual measured compliance governs useful credit. Eligible candidates undergo separate cold, network-disabled public and hidden scoring. Useful completion requires admission, nonempty passing checks and measured resource compliance. Failed, expired, unscored and incompletely validated cells remain in the denominator. Separate scoring alone does not establish resistance to arbitrary candidate interference.

The analysis averages repetitions within each family and then contrasts B with A across eight family blocks. A prespecified 20,000-resample paired family bootstrap provides a conditional descriptive interval, without a population superiority or noninferiority claim. Model, scorer and wrapper clocks may overlap and are not added as disjoint costs; unavailable usage remains unknown. The separate four-family pilot and earlier qualification controls supply no additional final trials. Appendices B, C and D retain the full design, interruption history, analysis recipe and accounting exceptions.

5 Historical cold-repair results

All 32 planned cells have verified terminal records. Nine yielded useful completion: A achieved 5/16 and B 4/16. The eight-family mean difference, B minus A, is -0.0625, with a descriptive 95% whole-family bootstrap interval of $[-0.1875, 0]$. Seven family differences are zero; Tornado accounts for the aggregate difference. One B Tornado request ended in a transport failure. This convenience sample supplies no population-level superiority or noninferiority conclusion and cannot isolate semantic content from other features of the combined request.

The 23 non-useful cells comprise 14 proposal-child deadlines, two known proposal failures and seven original hidden-failure labels. Six of the latter executed hidden checks and failed; one passed public checks but collected zero hidden assertions due to an import error. It remains incomplete validation with no useful credit. All failures remain in the fixed denominator. Sixteen scorer invocations include this incomplete case; fifteen completed cold validation. Provider usage is available for only 16/32 cells, and the original experiment has no complete-cohort token or settled-charge total.

The developmental four-family pilot remains separate, with ten historical useful labels among 24 cells, five per arm. Interruption, measured timing deviations, missing costs and reconstructed candidate metadata are retained in Appendix D. Those records preserve the original outcomes and do not provide additional scientific trials. The historical study supports an additive-context observation only; it does not evaluate the whole-system reasoning and reuse hypothesis.

6 Incremental system evaluation

The following evaluations extend the original architecture questions. They are development measurements and local qualification, separate from the historical cold experiment and from any future confirmatory population. Their profiles and source versions are explicit; a partial combined controller is qualified below, while the complete matched system comparison remains open.

Context substitution opportunity. We reconstructed all sixteen historical A/B request commitments across eight unique families and checked the selected public source against its hashes. All 254 selected windows were protected editable or test source; there were zero eligible unchanged-dependency files and zero substituted windows. The guarded substitution adapter therefore retained the raw source. Saved native packets contained no explicit obligation CIDs and 14,471 unresolved dependency-edge occurrences. Instruction text was retained, but this cohort cannot establish explicit obligation preservation or dependency compression.

Using `tiktoken` 0.12.0 with `o200k_base`, exact tokenization of system/user message content totaled 91,783 for A, 107,870 for B, and 108,577 for guarded substitution C: increases of 17.53% and 18.30% over A. These counts exclude chat-template overhead and output, and are not Grok provider usage. A second named encoding gives the same direction. Eighteen constructed adapter controls separately check expansion and rejection; they do not qualify a native producer. This result identifies a population requirement: a compression experiment must include unchanged dependencies that can actually be replaced under its stated policy.

Native dependency context and bounded projection. Three authored multi-module repositories supply six fresh native exact dependency admissions. Twelve cold/incremental index and semantic-bundle root comparisons agree; dependency edits invalidate prior capsules and unknown-call mutations require raw fallback. Full transported native capsules total 8,399 serialized context-content tokens versus 1,467 for raw source. A new restricted projection instead transports source-bound integer equations. Six separate Z3 equivalence checks and 252 concrete source executions check that projection; its 1,353 tokens are 7.77% below raw. Plain AST source compaction totals 1,464 tokens.

These are named-encoding offline counts, not provider savings. The equation profile excludes floats, custom objects, side effects and unsupported syntax. The compact projection is an evaluated addition; the full native capsule payload itself is larger than raw source in these examples.

Repository context and preparation. Native repository custody, current-source expansion and invalidation checks pass on bounded XML/pyparsing scopes and on both full declared Requests inventories (97 files each); these profiles permit zero raw-source omissions. Five full pyparsing producer processes reach their 600-second deadline. A subsequent immutable-state identity profile completes full preparation in 566.254 seconds, followed by a passing independent native consumer; all 302 XML/pyparsing regular files are retained and 22 negative controls pass. This establishes completion under the bound in one shared-host run. Preparation is still substantial, and these profiles establish no compression or replicated speedup. A subsequent full comment-change comparison exposes 1,568 differing graph records. A rescanning guard restores byte equality but fails a later import-inventory case. Preserving original frontend observations then gives exact full equality while reusing 153 of 154 Python files. Incremental takes 607.971 seconds versus 440.276 cold, adds a 53.75 MB sidecar and incurs separately metered bootstrap costs. Useful incremental economics remain open. Appendix G retains source scopes, identity checks, earlier failures and actual costs.

Live context substitution. The three dependency-bearing development repositories also support a matched raw-versus-equation repair pilot. Both arms pass all three separate cold behavioral checks; all six requests return usage. Provider input totals are 4,014 raw versus 3,817 compact (4.91% fewer), and total usage is 4,866 versus 4,637 (4.71% fewer). Both arms use three requests with the same Grok 4.6 low-effort profile and resource ceilings as the routing pilot. Cached-input totals differ, so charge differences cannot be attributed solely to compaction. This tests a restricted, verified representation on trivial constructed repairs; dynamic source expansion and broader repository quality remain unmeasured. The result is separate from, and must not be added to, the routing reduction.

Checked symbolic closure and live residuals. A new development caller invokes the native Python analyzer, bounded CEGIS repair, visible behavioral checks and native routing. Four constructed tasks include false success without observation, swallowed backend failure, interval clamping and stable deduplication. Both arms receive the same source, objective and public checks; separate Docker execution scores broader behavioral checks. The first two defects fit the declared deterministic grammar. The remaining two require the same model residual in both arms. Internal CEGIS marker gates are not counted as theorem proofs; behavioral checks justify only these bounded obligations.

Table 1: Constructed four-task routing development comparison. One observation per task and arm; no population inference.

Arm	Independently passed	Model calls	Total provider tokens
Model first	4/4	4	4,865
Symbolic first plus residual	4/4	2	2,430

All six Grok 4.6 requests returned usage, under the same low-effort profile, `max_tokens=2048` request parameter and 240-second deadline, without retries. The observed reduction is 50.05% on this suite. Eliminating two requests accounts for 2,282 baseline tokens; response variation on the residual tasks accounts for the remaining 153-token net difference. Provider cache state was not controlled. There were no auxiliary workflow model calls or model-assisted setup in this caller; experiment authoring and implementation effort are outside that runtime denominator. The initial expired-credential attempt dispatched zero requests and remains a separate development record. General repository performance and full-system setup amortization remain unmeasured.

Supplementary scorer validation. A post-development audit exposes five false admissions among six crafted incorrect candidates in the original pilot scorers. Candidate interference with oracle helpers or premature process termination could masquerade as passing execution. A new isolated scorer separates the oracle from the candidate worker, restricts public dependency imports and requires observed calls and positive completion. It rejects all six crafted candidates and all seven unrepaired controls. All ten unique saved candidate bodies still pass, preserving the eight routing and six context

record outcomes above. This supplementary rescore supports these saved outcomes; the small control set does not establish general scorer robustness. Original scorers and observations remain retained.

Lifecycle-preserving test reuse. A proposed local evaluation adapter uses native cache admission and Ed25519 runner attestations issued only after actual passing setup/call/teardown. Its explicitly enrolled pure workload and immutable fixture profile support valid call-phase reuse while setup and teardown still execute. Fourteen prespecified cases cover unchanged work, source/test/fixture/configuration/ environment mutations, missing or tampered evidence, revocation, unsupported fixture values and lifecycle failure. All fourteen match separate cold-process outcomes. Reuse followed by failing teardown correctly fails the whole item; the result is not zero-error evidence for arbitrary pytest programs.

Five alternating warm/cold timing pairs omit all five warm test-body executions while retaining five setups and teardowns in each arm. Total host wall time was 1.281 seconds cold and 1.240 seconds warm, excluding the separately recorded prime and issuance costs. This small descriptive difference does not establish a reliable net speedup or token savings. The default production plugin probe executed the body in both write and read/write modes and produced no positive reuse. Thus the result qualifies the proposed bounded adapter, not deployment of the native default reuse path. Earlier failed development invocations and the historical false-reuse observation remain retained. The measured 0.256-second cold prime and 0.006-second issuance exceed the observed wall-time saving over these five repetitions; a net benefit at that horizon is not demonstrated.

Persistent work across process termination. Three actual processes exercise the native session journal/checkpoint with the same checked deterministic caller. Duplicate pending notifications coalesce. The first process exits abruptly after one completion; the next resumes exactly one unfinished scan, and a third restart finds no pending work. Each actual execution passes the independent behavioral oracle, and no original attempt executes twice. However, a fresh notification for a completed snapshot schedules and executes analysis again. Journal continuity is supported by this case; completed-result memoization, accepted-root publication and cross-task token savings are not established.

Publication and recovery qualification. Forty-seven existing source-shipped tests pass in a fresh compatible native profile. They cover immutable-root publication, stale writers, competing CAS updates, seal storage and restart recovery at injected persistence boundaries. These are constructed software qualifications, including placeholder proof records and exception-based interruptions. They do not establish a newly integrated evidence-producing workflow, actual process-kill tolerance at each boundary, or measured parallel-sealing savings.

Evidence-backed sealing and completed recovery. A new caller independently replays finite computation receipts, checks exact source/input/policy bindings and complete coverage, then invokes native integrity-only seal verification and publication. A full checkpoint and two delta seals become successive current roots; a competing delta loses stale parent CAS. Four prepared states cover 700 independently checked state/input observations, using five computation subprocesses and three unchanged-receipt reuses. Five constructed mutations reject. Native digest verification alone accepts three internally consistent but semantically wrong or stale receipts; the caller's independent replay and bindings are therefore substantive gates.

Two real process terminations preserve the expected accepted roots. Native recovery recognizes a valid pre-CAS seal but leaves its transition pending; retrying the same publication is refused. A separately evaluated recovery caller rechecks the persisted evidence and completes native CAS/WAL phases with zero new computation calls. A subsequent fresh process consumes that accepted root and publishes the next changed state, reaching a four-seal chain. This establishes bounded continuation under an explicit added caller. It does not establish native automatic recovery, theorem proofs, power-loss tolerance or a compute saving: the independent receipt checker still runs on reuse. Four paired trials separately compare actual preparation of two cold operations: all eight preparations pass with identical receipt and seal content. Median preparation time is 63.21 ms sequential versus 32.10 ms parallel; maximum sampled aggregate process RSS rises from 44.66 to 55.54 MiB. The host is not exclusive, sampled RSS counts shared pages per process, and these tiny preparation trials measure neither publication latency nor a whole-system speedup.

Procedure synthesis and stored candidates. A bounded native synthesis/body-store comparison completes seven of eight units in both cold and stored arms: 3,883 correct fresh-query decisions, zero false admissions and 554 abstentions on unsupported parity. Stored candidates require six synthesis calls and 23 tried candidates versus eight and 29 cold, under exact source-catalog and policy bindings. A separately frozen fixed-template control matches quality with no synthesis, so retrieval has no demonstrated advantage over that simpler control. Fresh-process replay, binding mutations and scaffold sensitivity are checked; this initial executor/store profile has no model calls. A later signed native TEST integration executes the exact saved candidate on 136 new bounded inputs through all six native handlers. All decisions match the oracle; 30 evidence negatives and 13 runtime controls pass. Its 17.243-second worker cost plus preparation does not establish savings. Appendix J separates this tested execution from repository applicability and promotion.

World-state memory and actual subsequent actions. A native typed transition producer, persistent block/SQLite store and admission path are connected by a new evaluation dispatcher. Eleven tasks per arm run in fresh interpreters against a closed three-action integer service; selected actions always execute and their current outputs are independently checked. All 44 task/arm outcomes pass. Native memory uses 16 action executions versus 27 without memory, but exact JSON retrieval and complete-condition templates also use 16. Charging each arm the common three-action seed gives 19 versus 30. Changed state, source and declared environment invalidate prior observations; fallback creates fresh evidence consumed by the next process. Native total process time is 8.947 seconds versus 8.739 without memory, including common import/storage work and excluding the separately recorded shared seed. Thus fewer search attempts do not establish a wall-time advantage. The frozen dispatcher uses retrieved typed records directly: an audit finds that the native compiler emits incorrect record identifiers in all eleven compiled proposals, so compiler-mediated dispatch is not qualified by this run. These results establish bounded storage/lookup consumption, not learned world-model benefit, transfer, production integration or token savings. A separately frozen compiler correction selects each record's own identity. The ensuing consumer resolves compiler-selected records from the native store before actual execution: the seed and eleven later cases pass, using 19 actions and 25 fresh service processes. Changed source, state and environment still invalidate prior observations. Independent checks preserve all original comparison artifacts. This qualifies the corrected compiler-mediated path; it does not change the original simple-retrieval/template parity or supply learned transfer and token-savings evidence.

Native compositional reasoning. An existing original-source vertical slice was run with its compatible nested datasets tree. A baseline component contract discharged, an injected fault was refuted, and repair restored discharge. Localization produced an UNSAT core and a cvc5 interpolant separately checked by Z3; the final artifact was verified by semantic rescanning, composition redischarge and test replay. Selected and full tests passed. Host elapsed time was 19.309 seconds. This is one existing constructed fixture with a prescribed fault and expected repair, not a held-out synthesis experiment. Internal benchmark estimates are not counted as measured token or compute savings. An earlier source profile failed to import the required composition module and is preserved separately. Solver validation does not imply kernel reconstruction or all-language correctness.

Native proof production and statement binding. The source-built native backend produces three v3 Merkle/count proofs and three honest v2 implication-chain proofs. For v3, primary setup takes 7.810 seconds, proving 12.048–12.474 seconds and verification 3.053–5.147 milliseconds. Tested malformed witnesses, proofs and keys reject. Both profiles require exact outer-statement checks at the caller: the raw verifier can accept changed displayed metadata when its scalar public inputs remain unchanged. These are measured bounded circuit relations, not proofs that arbitrary tests executed or production trust admission. Stronger v2 textual-binding assurance remains unconfirmed, and the v5 pytest-attestation profile is unsupported. Appendix I retains the separate profiles, mutation observations and v2 costs. A subsequent native archive producer/consumer binds four actual accepted controller events to one honest v3 proof, with seven mutation rejections. A second integration consumes the actual accepted Requests repair, restores all 97 source files and rejects eight mutations. Its 11.979-second proof and 5.256-millisecond verification are post-cohort overhead; original receipts still supply execution evidence.

Combined controller and native publication. Constructed thirteen-event reasoning and eight-event publication profiles join the actual native services and exercise real interruptions, native

full/delta publication and fresh accepted-source consumption. A separate Requests curator fixture measures 377.226 seconds of phases including full context preparation. These zero-provider controls establish bounded continuation, not token savings. Appendix H retains their complete costs, failures and unsigned integrity-only trust boundary.

Natural two-issue comparisons. Two frozen Requests 5414-to-6028 development cohorts compare four arms with equal public access and four-call trajectory ceilings. Every arm in each cohort accepts the first issue and fails the second. First requests within each cohort are byte-identical; B permits no source omission, while C/D establish no successful symbolic closure. Partial D publishes and retrieves accepted source. The first cohort's A/C/D continuations exceed their source bound; B spends its remaining calls on failed candidates. A new conflict-first retriever keeps required source within the same bounds in the second cohort. All sixteen calls then execute, including expansion requests and three outputs rejected by a file-mode transport defect. Table 2 retains both cohorts separately. In the second, B/C/D use 10.43%/12.01%/7.77% more tokens than A at the same observed acceptance count. Uncontrolled server caching, one exposed family and adaptive follow-up preclude causal component or general efficiency claims. Complete-trajectory cost ratios remain undefined.

Table 2: Natural Requests cold development cohorts, retained separately. Every arm accepts 1/2 nodes and completes 0/1 trajectories in each cohort. Second-cohort wall includes nested services; first-cohort timing limits remain in the appendix.

Arm	First calls	First tokens	Second calls	Second tokens	Second seconds
A	1	14,428	4	56,543	227.882
B	4	65,695	4	62,439	249.385
C	1	14,589	4	63,336	415.743
D (partial)	1	14,269	4	60,937	489.516

Remaining system questions. These increments establish local mechanisms and expose limitations. They leave open matched context-substitution efficacy, persistent multi-task quality/cost, procedure and world-memory utility on real tasks, production reuse, governed publication, integrated native proof admission, and a held-out comparison with complete startup, fallback, failed-attempt and recovery accounting. Additional benchmark recruitment and environment qualification are reported in Appendix F; they add no model-quality outcomes.

7 Related work

Proof-carrying code places explicit evidence between a producer and a consumer's safety policy [7]. This motivates separating an artifact from the evidence needed to accept it. Our finite-check repair experiment and signed provenance records are not proof-carrying code in that formal sense. Abstract interpretation and translation validation similarly motivate explicit approximation and conversion relations [2, 10], not automatic admission of a backend or source-language proof.

Table 3: Useful completions in the fixed 32-cell A/B cold comparison. Eight families are the analysis blocks; two repetitions per arm are nested.

Family	A useful / 2	B useful / 2	B−A mean
09-tomlkit	1	1	0
10-installer	0	0	0
11-tornado	2	1	-0.5
12-more-itertools	2	2	0
13-charset_normalizer	0	0	0
14-iniconfig	0	0	0
15-wheel	0	0	0
16-jinja	0	0	0
All eight (16 cells per arm)	5 / 16	4 / 16	-0.0625

Mokhov et al. separate scheduling from the decision whether a task needs rebuilding [4]. Our reuse obligation additionally names the statement, environment, lifecycle, and trust conditions for transferring a test observation. The cold comparison supplies no measured incremental-reuse advantage over build systems.

SWE-agent studies an interface for repository navigation, editing, and program execution [13]; SWT-Bench studies test-based validation of real-world fixes [5]. They motivate treating both information access and executable checks as parts of a repair method. Differences in tasks, interaction, models, and acceptance criteria preclude a head-to-head performance claim or an assertion that we reran their benchmarks. The contribution here is a bounded composition of evidence and authority mechanisms, not a first-of-kind claim or a new foundational algorithm.

8 Limitations

The historical study's eight convenience-selected families and two nested repetitions cannot support broad repository-population inference. Its intervention changes the complete context supplied to the model; component ablations and a prompt-compression causal claim remain unavailable. The withdrawn C/D, warm-cache, reuse, publication, and sixteen-family objectives are not completed outcomes. The new context and routing pilots use small constructed tasks and distinct source profiles. Their observed reductions cannot be added together or extrapolated to an integrated system. Qualification counts and exhaustive bounded input grids are not independent samples of repository tasks. The memory comparison is one constructed sequence; ordinary retrieval and templates match its action-count reduction. Sealing measurements use finite execution receipts, explicit added admission/recovery callers and two tiny preparation operations. These profiles have not yet been combined into a single matched repository-trajectory experiment.

Finite tests cover their executed assertions. Passing them does not establish complete behavioral correctness, source-level semantic fidelity, or adequate coverage of every relevant behavior. Independent human semantic annotations, expert agreement, and human time were not measured. AI candidate inspection does not supply those measurements.

The conditional design arguments retain checker adequacy, complete dependency projections, protected policy authority, and exclusive checked publication as premises. They establish no universal Python theorem, native proof performance, global consensus, external exactly-once effects, or complete adversarial scorer integrity. A local constructed witness cannot fill a missing production or final-comparison measurement.

The interrupted runtime, actual timing deviations, prior failures, and unknown remote charges limit net-efficiency claims. Lease occupancy is not compute usage. Overlapping clocks and a pilot cost projection that aliases the same executions cannot be charged twice. Separate source epochs and actual cap observations must remain visible even when a candidate passes its tests.

Numerical reproduction from anonymous, manifest-bound scalar derivatives is narrower than original-signature verification or confidential-oracle rescoring. Changed derivative bytes are not authenticated by the original signature. Reference source derivatives must be distinguished from executed source, with upstream attribution and licences retained. Stable provenance hashes can permit correlation, so a lexical anonymity audit is bounded and does not guarantee de-identification.

9 Conclusion

Persistent semantic state, obligation-guided reasoning, applicable evidence reuse and accepted-state publication provide a method for avoiding repeated work under a declared acceptance standard. The development experiments exercise these mechanisms and expose their costs: symbolic closures avoid model calls on constructed tasks, simple controls match some memory benefits, and priming can outweigh validation reuse. Native publication and archive proofs preserve declared bindings without proving arbitrary execution truth. Neither natural repository development comparison completes a whole trajectory or establishes a general efficiency advantage. The earlier additive-context study remains a separate, unfavorable result. Complete system-level token savings require matched, independently scored trajectories, mechanism ablations and full accounting of setup, failed attempts

and recovery. The architecture defines this research question; the missing integrated measurements remain necessary.

References

[1] Edmund M. Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. Counterexample-guided abstraction refinement. In *Computer Aided Verification (CAV 2000)*, volume 1855 of *Lecture Notes in Computer Science*, pages 154–169. Springer, 2000. doi: 10.1007/10722167_15.

[2] Patrick Cousot and Radhia Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, POPL '77, pages 238–252, Los Angeles, California, 1977. ACM Press. doi: 10.1145/512950.512973.

[3] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL `https://openreview.net/forum?id=VTF8yNQM66`.

[4] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build systems à la carte. *Proceedings of the ACM on Programming Languages*, 2(ICFP):1–29, 2018. doi: 10.1145/3236774. Expanded as Build systems à la carte: Theory and practice, Journal of Functional Programming 30, e11, 2020, doi:10.1017/S0956796820000088.

[5] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. SWT-bench: Testing and validating real-world bug-fixes with code agents. In *Advances in Neural Information Processing Systems 37*, pages 81857–81887. Neural Information Processing Systems Foundation, 2024. doi: 10.52202/079017-2601.

[6] Nebius. SWE-rebench dataset. `https://huggingface.co/datasets/nebius/SWE-rebench`. Test-split metadata at revision 89cdfbab; accessed 2026-09-14.

[7] George C. Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '97, pages 106–119, Paris, France, 1997. ACM Press. doi: 10.1145/263699.263712.

[8] OpenAI. Introducing SWE-bench verified. `https://openai.com/index/introducing-swe-bench-verified/`, 2024. Accessed 2026-09-14.

[9] OpenAI. Why SWE-bench verified no longer measures frontier coding capabilities. `https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/`, 2026. Published February 23, 2026; accessed 2026-09-14.

[10] Amir Pnueli, Michael Siegel, and Eli Singerman. Translation validation. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 1998)*, volume 1384 of *Lecture Notes in Computer Science*, pages 151–166. Springer, 1998. doi: 10.1007/BFb0054170.

[11] Scale AI. SWE-bench pro dataset. `https://huggingface.co/datasets/ScaleAI/SWE-bench_Pro`. Test-split metadata at revision 7ab51149; accessed 2026-09-14.

[12] Armando Solar-Lezama. *Program Synthesis by Sketching*. PhD dissertation, University of California, Berkeley, 2008. URL `https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-176.html`. Technical Report UCB/EECS-2008-176.

[13] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37*, pages 50528–50652. Neural Information Processing Systems Foundation, 2024. doi: 10.52202/079017-1601.

A Full conditional argument record

The following audited statements are retained verbatim from the earlier formal argument record. Their Equation 2, Algorithm 1, and statement identifiers name that record; they are not additional final experiments or numbered algorithms in the main text. References to a sealed profile or the absence of a campaign describe the scope of that argument audit. Later constructed corrections and the cold comparison do not retroactively discharge its universal premises. The intended algorithm's bounded retry branch is architectural specification; the final empirical protocol permits no proposal retry.

Conditional acceptance and reuse arguments (audited)

The statements below restate Equation 2, the accepted-lineage sketch, the dependency-projection reuse obligation, the three acceptance questions, and Algorithm 1 with explicit scope, premises, and conclusions. Every sketch is **unmachine-checked**: the sealed validation profile has no usable `z3`, `lean`, `coqc`, or `cvc5`, and no independently checked proof artifact discharges these arguments. Implementation evidence supports only the named tested premises. External side-effect atomicity and universal dependency closure are not inferred. Unknown specifications stay visible.

Statement FA-EQ2 (acceptance invariant)

Scope. One publication of an accepted lineage root under a declared artifact policy. The statement is a design invariant for that publication operation. It is not a theorem that every current write path enforces the invariant, and it is not a claim about Git, remote stores, or physical effects outside the tested expected-parent compare-and-swap.

Premises. Let s be the current accepted root, s' a candidate successor, a the requesting actor, and q the required obligation set. Write $\mathrm{Publish}(s')$ for the unique accepted-lineage publication of s'.

Conclusion.

$$\begin{aligned}\mathrm{Publish}(s') \Rightarrow\ & \mathrm{CurrentParent}(s) \wedge \mathrm{Authorized}(a, s) \\ & \wedge \mathrm{Scoped}(a) \wedge \mathrm{CompleteManifest}(q, s') \\ & \wedge \mathrm{EvidenceAdmitted}(q, s') \wedge \mathrm{CAS}(s, s').\end{aligned} \tag{2′}$$

Argument (unmachine-checked). If publication is defined to succeed only after those six gates, then any published s' satisfies them. The converse is not claimed: a candidate that meets the gates may still lose compare-and-swap. Failed, stale, incomplete, or simulated attempts are not $\mathrm{Publish}$ successors.

What the invariant does not establish. Properties omitted by the checked policy; truth of unsupported or lossy translations; honesty of signed observations; atomicity of external side effects; global consensus; or universal enforcement by unexamined write paths.

Statement FA-I31 (accepted-lineage induction)

Scope. Roots reachable *solely* through accepted publication transitions of the operation in FA-EQ2, under a fixed statement profile Θ and policy predicate P_Θ over artifact manifests.

Premises.

1. **Initial-root policy.** The initial accepted root satisfies P_Θ.
2. **Exact input.** Required subject bytes and dependency references are verified before admission.
3. **Complete manifest.** The requirement manifest is complete under an admitted completeness rule; membership of some units is not completeness.
4. **Checker.** Each accepted evidence item satisfies its designated checker relation and evidence class.

5. **Translation bound.** Lossy or unsupported translations cannot discharge source-level obligations.
6. **Current parent.** Publication names the expected current parent.
7. **Exclusive publication.** Compare-and-swap linearizes concurrent same-parent writers; losers are not successors.
8. **Protected authority.** Policy and checker identities cannot change under ordinary worker authority.

Conclusion. Every root reachable solely through accepted publication transitions satisfies $P_{\ominus}$.

Argument (unmachine-checked). Induction on the number of successful publications. The base case is premise 1. For the inductive step, the publisher checks $P_{\ominus}$ on the exact candidate manifest and current parent before compare-and-swap linearization. Failed, stale, incomplete, or simulated attempts are excluded from the successor relation, so they are not inductive cases. The induction preserves only properties actually expressed by $P_{\ominus}$.

Nonvacuity. The successor relation is nonempty on the tested sealer: a complete current manifest can publish, while an empty or incomplete mandatory manifest cannot. Deny-all publication would make the invariant hold vacuously and is not a useful-progress witness.

Statement FA-I32 (dependency-projected test reuse)

Scope. Transfer of a previous setup/call/teardown observation of test t between states s and s' under a declared runtime/effect profile θ. The statement is a proof *obligation*, not a theorem of arbitrary pytest execution.

Premises. Let $\mathrm{Obs}_\theta(t, s)$ be the relevant observation. Suppose there exist a complete projection π_D and a deterministic function f_θ such that

$$\mathrm{Obs}_\theta(t, s) = f_\theta(\pi_D(s))$$

on every state admitted by the reuse policy.

Conclusion. If $\pi_D(s) = \pi_D(s')$, then $\mathrm{Obs}_\theta(t, s) = \mathrm{Obs}_\theta(t, s')$. An admitted previous observation may transfer only inside that profile and those assumptions.

Argument (unmachine-checked). Immediate from functionality of f_θ. The difficult premises are completeness and determinism of π_D. Equal digests of incomplete manifests do not satisfy π_D completeness. Opaque, uncontrolled, or unknown fixture values, incomplete runtime traces, and teardown failures do not discharge the obligation; they require execution or fail closed. A cold full-run oracle and adversarial mutations can check an implementation; they do not prove universal dependency closure. No such independent cold-oracle campaign is recorded here.

Statement FA-I33 (three acceptance questions)

Scope. Any candidate action that might be described as "accepted."

Premises and conclusions (separate).

Question	What a positive answer establishes, and what it does not
Behavioral claim	The candidate program or model satisfies its stated obligation under the declared checker, environment, and evidence class. This is not permission to act and not publication of a lineage root.
Action permission	This actor may perform this action now under the bound lease, policy, scope, and roots. This is not behavioral correctness and not publication.
Publication	This candidate and its evidence become the current accepted root. This is not, by itself, proof that omitted obligations hold or that external effects occurred exactly once.

Neither a proof of behavior nor a capability grant answers all three. Formalization fixes the statement; authorization binds actor, tool, arguments, resources, effects, scope, time, and roots; execution observations report what happened; publication chooses the accepted lineage.

Statement FA-ALG1 (composed supervision loop)

Scope. Algorithm 1 remains the intended composition: a control-flow specification, not a newly executed full-loop result. It is labeled **executed composition** only on the steps supported by the integrated guard-before-write witness. Passing isolated stop-path tests does not establish the complete loop.

Intended algorithm. Input: authorized query q, current source forest F, policy P, budget B.

1. Reconcile event, task, and artifact state; refresh F and derive G, O.
2. If cancelled or budget exhausted: return explicit non-success.
3. If every current obligation is admitted: return goal-completion verification.
4. Generate bounded task proposals; preview and separately admit ready work.
5. If no task is admissible: return wait-or-review, not goal success.
6. Claim the task; bind lease/fence, preimages, required context, and validation.
7. Choose a deterministic, authorized residual-model, review, or unavailable route.
8. If review or unavailable: record a blocked attempt; return to the event service.
9. Produce and apply a candidate only through checked isolated execution.
10. Rescan; derive O_{new} from authorized rules, not the worker's pass claim.
11. Verify or reuse evidence; retain failed, unknown, and pending obligations.
12. If the failure is refinable: record a new residual; return a bounded retry.
13. If evidence is incomplete or unsafe: return defer-or-reject.
14. Persist the manifest; recheck current bindings; compare-and-swap the expected parent.
15. If publication conflicts: return reconcile, not completion.
16. Record the accepted task transition and updated F, G, O; reevaluate at line 1.
17. Propose procedure or policy learning separately; never self-promote.

Executed composition (integrated witness only). The integrated witness supports a frozen guard-before-write cycle: authorized pre-state and obligation; counterexample; admitted deterministic route; isolated patch application; independent tests; narrow cache reuse of an unchanged eligible receipt; complete-manifest publication; and rejection of invalid, incomplete, and simulated non-completion. Those steps correspond to intended lines 4, 7 (deterministic), 9, 11 (narrow reuse), 13, 14, and 16, plus the negative reading of line 17 (no worker self-promotion). They do *not* execute the live event-service loop, residual production-model dispatch, budget-exhaustion composition, wait-or-review, procedure/policy learning, native proof, federation, or a matched A–D experiment.

Trusted limitations retained with the algorithm. Unknown or inconsistent specifications remain visible. Incomplete projections cannot silently authorize reuse. External exactly-once behavior outside the tested write-ahead-log / compare-and-swap / lifecycle boundary is not claimed. Optional native proving and world/procedure extensions are out of scope unless a later task supplies an eligible profile and consumed-result evidence.

B Frozen comparison

Population and treatment. The eight recruited families are Tomlkit, installer, Tornado, more-itertools, charset-normalizer, iniconfig, wheel, and Jinja. They are a convenience sample, not a representative sample of Python repositories. Two arms and two nested repetitions yield 32 fixed cells, 16 per arm. Repetition identifiers 104729 and 130363 identify design positions; they were not served to the model API as random seeds.

Before final outcomes, the A/B local-cold comparison was retained as an order-preserving subset of the original 192-cell plan. The remaining 160 factor cells were withdrawn, and eight additional target families were not recruited. C/D routing, warm caches, test reuse, publication efficacy, the original sixteen-family inferential objectives, and D/A promotion comparisons remain outside this experiment. The narrowed comparison is descriptive.

Arm A receives the selected public raw context. Arm B preserves the matched raw selection and adds actual native source-linked semantic context. Sixteen exact family/arm request commitments precede final generation; each arm's two repetitions use its fixed request. This treatment changes the combined request and cannot isolate semantic subcomponents or prompt compression. Public source and native preparation remain separate from confidential acceptance tests.

One proposal and independent cold execution. Each cell permits one Grok 4.6 proposal POST, requesting at most 4,096 output tokens through a strict structured edit response. There is no scientific fallback, retry, served seed, or reasoning-effort parameter. A failed request consumes its original cell rather than creating a replacement. The frozen order is executed sequentially.

Proposal execution uses one CPU, 2 GiB, and a 32-process limit. The nominal gateway and HTTP-wrapper bounds are 600 seconds; the child bound is 595 seconds and the socket timeout is 590 seconds. Compliance depends on actual measured intervals. A configured limit, missing measurement, or nonfinite value is not evidence that the budget was met.

The operator retains exact candidate bytes and a specific AI review before eligible scoring. That procedural review supplies no independent human annotation. Scoring runs separately in a cold, network-disabled container with one CPU, 2 GiB, and a 128-process limit, bounded by 120 host seconds and 115 child seconds. The scorer's process limit was fixed prospectively for both arms and all eight families because unchanged public concurrency tests require up to 100 threads. The earlier 32-process public-baseline failure remains recorded; proposal and native-preparation limits remain 32.

Useful completion requires an admitted candidate, nonempty public and hidden checks all passing, and measured resource compliance. Failed, refused, expired, unscored, or incomplete cells receive no useful-completion credit. Unknown states remain explicit and prevent a complete-cohort analysis until their terminal accounting is resolved. An import or collection error with no hidden assertions executed is incomplete validation, not an observed hidden behavioral assertion failure. Separate scoring and sealed records do not establish comprehensive resistance to adversarial scorer manipulation.

Interruption and provenance. The original resource lease ended during cell 7. Its request, failure, cleanup, and unknown termination-sender evidence remain retained. A separately admitted managed lease and continuation runtime govern cells 8–32 without replaying the consumed prefix or changing requests, model, order, scorer, limits, or analysis. Both runtime source versions remain explicit. The lease is a cooperative allocation, not an exclusive physical resource reservation; per-phase process boundaries apply the configured resource limits.

Continuation checks add work around effect admission and receipt signing. The signed gateway interval ends before signing; whole operator invocation time extends through subprocess completion and output flush/synchronization. These distinct intervals must retain their endpoints rather than being presented as a single total host clock.

C Analysis and evidence separation

For family f, arm a, and repetition r, let Y_{far} indicate admitted useful completion. Average the two fixed repetitions within each family and arm: $\bar{Y}_{fa} = (Y_{fa1} + Y_{fa2})/2$. The retained contrast is

$$\widehat{\Delta} = \frac{1}{8} \sum_{f=1}^{8} \big(\bar{Y}_{fB} - \bar{Y}_{fA}\big).$$

Families are the independent analysis units; 32 nested cells are not 32 independent repositories. A descriptive robust-family outcome additionally requires both repetitions to succeed and is not a promotion decision.

The frozen bootstrap draws eight whole family blocks with replacement, sharing each draw across arms and repetitions. It uses 20,000 resamples, Python 3.12 `random.Random` with seed 20260911, lexicographically ordered family identifiers, and exactly eight `randrange(8)` draws per resample. The 95% percentile endpoints use linear interpolation at $(B-1)p$ for $p \in \{0.025, 0.975\}$ and $B = 20{,}000$. These intervals are conditional, descriptive summaries of eight convenience-selected families, not population-level coverage or superiority/noninferiority decisions.

Failures remain in the fixed outcome denominator. Resource summaries report their available-observation counts and missingness. Provider child, HTTP wrapper, gateway, scorer, and operator clocks can overlap and are not added as disjoint costs. Shared native preparation is counted once. Baseline checks, prior failed attempts, review, queue waits, cleanup, and artifact work retain their own accounting scopes. Token usage and cost ticks are telemetry; unavailable settled charges and unmeasured effort remain unknown.

The developmental pilot is a separate four-family, 24-cell A/B study with three nested repetitions. Its retained labels include ten useful completions, five per arm, from 24 POSTs and 12 scores. Ten child durations exceeded 595 seconds by less than 1 ms, and pilot cell 20 exceeded 600 seconds in both wrapper and gateway clocks. These deviations and labels remain preserved; the final resource policy is not applied retrospectively to relabel the pilot.

Earlier boundary evidence is also separate from final repair outcomes. The mutation/cold comparison retains one false reuse among all 26 cases, rather than reporting a selected zero-of-25 subset. All four later correction invocations, including failed and partial attempts, remain visible; corrected full-cold execution supplies no positive reuse credit. Constructed gate checks and public-baseline passes do not count as useful final repairs.

D Historical cold-study accounting details

Complete fixed-cohort accounting. All 32 planned cells have verified terminal records; none is missing or replaced. The experiment made 32 original proposal POSTs and 16 scoring invocations, of which 15 completed cold validation. Useful completion was 9/32 (28.125%): A achieved 5/16 and B 4/16. Table 3 reports the family counts. Failures remain in these denominators. The mean of the eight paired family differences, B minus A, is -0.0625, or -6.25 percentage points. The frozen 20,000-resample whole-family bootstrap gives a descriptive 95% percentile interval of $[-0.1875, 0]$. Each resample draws eight families and preserves both arms and their nested repetitions. The interval describes this convenience sample; it supplies no population-level superiority, noninferiority, or promotion decision.

Concentration across families. Tomlkit yielded one useful completion per arm, Tornado two for A and one for B, and more-itertools two for each arm. The other five families yielded none. Seven family differences are zero; the Tornado difference is -0.5 and accounts for the aggregate gap. Its B cell 012 (repetition 130363) ended in a transport failure without a response or scorer invocation. This outcome remains part of the fixed comparison. The data do not isolate an effect of semantic content, a particular native component, or prompt compression. The treatment changes the combined request, and the recorded endpoint includes operational and provider failures.

Failure categories and incomplete validation. The remaining 23 cells comprise 14 proposal-child deadlines, two known proposal failures, and seven records with the original signed label `hidden_acceptance_failed`. The two proposal failures were cell 003 (Tomlkit, A, 130363), which returned HTTP 503, and the transport failure in cell 012. Their known local termination does not establish whether a remote charge occurred. Among the seven hidden-failure labels, six completed cold validation and failed executed hidden checks. Cell 005 (installer, A, 104729) instead passed 28/28 public checks but collected zero hidden assertions and ended with an import-name failure. It is incomplete hidden validation, neither an observed hidden behavioral assertion failure nor a demonstrated external infrastructure fault. Its original label and zero useful credit are preserved; no candidate, test, or failed request was revised and rerun.

Measured time and unavailable costs. The provider-child, HTTP-wrapper, and signed proposal-gateway clocks each cover 32/32 cells. Their medians are 472.777, 473.660, and 474.107 seconds, respectively. The 14 deadline cells are resource-unqualified under the frozen measured predicates;

configured caps do not imply that every measured duration complied. Scoring-gateway time is available for 16/32 cells, with a median of 2.500 seconds across those invocations, including the incomplete validation. Only one of eight families has complete scoring-time observations for both repetitions of both arms. The frozen policy therefore leaves the paired scoring-cost interval unavailable instead of imputing zeros or bootstrapping fewer families. Grant-to-terminal duration includes waiting and bookkeeping: its median is 603.644 seconds and its maximum 10,938.574 seconds. These intervals overlap and are not summed. API usage is reported for 16/32 cells; settlement is unavailable for all 32. No dollar total, complete effort cost, or net efficiency gain is established.

Runtime and accounting deviations. The original lease ended during cell 007. Its consumed failure and unknown signal sender remain recorded; a prospectively admitted continuation covers only cells 008–032, with separately identified executable sources. The original requests, order, limits, and analysis policy remain fixed. The 26 additive disclosures also preserve pre-effect launch errors, bookkeeping takeovers, diagnostic corrections, and interrupted operator work. Cell 027 had sufficient credential headroom at its metadata check but only 895.290 seconds at the later reservation, below the intended 900-second margin; exact POST time is unobserved. Cell 029's delayed bookkeeping enlarges its grant-to-terminal clock without creating another scientific trial. Outer invocation durations and work after signed-gateway clock sampling retain separate scopes and missingness. They are not added to the signed gateway interval or treated as measured AI reasoning effort.

Evidence and comparison limits. Fifteen failed proposals lacked original candidate-binding files. Their later unsigned sidecars describe the unchanged public inventory and an empty patch; they are derived metadata, not returned model edits, recovered original files, or successful candidates. The four-family developmental pilot remains separate: its 24 cells yielded ten historical useful labels, five per arm, with retained timing deviations rather than retrospective final-policy compliance. Earlier boundary evidence in Table 6 is also separate from final repair outcomes. The ablation scope in Table 7 records unmeasured or withdrawn endpoints. C/D routing, warm-cache reuse, publication efficacy, and the original sixteen-family inferential objectives are not evaluated by these 32 cells.

E Explicit public development objectives

Twelve newly authored public examples make three original development objectives concrete using their retained public titles and prefix-source APIs. They are not recovered historical issue text, hidden acceptance checks or independent families. On unchanged XML, OAuth and Click prefixes, respectively, 2/4, 1/4 and 2/4 examples conform. XML attaches namespace declarations to a descendant when the declaring element has no ordinary attributes; OAuth authorization metadata raises an AttributeError, including with explicitly supplied response types; Click help still enumerates choices when their display is disabled. Public comparison cases exercise ordinary attributes, disabled namespace processing, token-only metadata, visible choices and explicit metavar behavior.

All twelve examples run in three fresh processes under frozen 60-second bounds, with no setup errors. An earlier run's three missing-constructor-argument errors are retained as harness failures. The revised OAuth setup uses actual token and endpoint classes and all required combined-server claims. Full unchanged source inventories retain 16, 199 and 145 entries; actual imported modules bind to those prefixes. A separate 545-check audit reconstructs predicates and source/process provenance without further execution. The new objective manifest binds text, examples, repair surfaces and public API sources. Together with the separately retained nine-example Pyparsing public-report reproduction, these provide explicit development objectives for the original four-family matrix. Independent scorer admission, complete operational trajectories and matched cache/component arms remain required. No model or hidden scorer is called; example conformity is not a repair-success or system-quality estimate.

A later qualification exposes a limit in the newly authored OAuth expectation. The original accepted reference passes three of four public examples; combined server metadata omits one configured response type. An independently authored configured-key-preserving candidate passes all four new examples but fails the unchanged original acceptance check. Public-only API review does not justify requiring the abstract-validator metadata probe to advertise every internal dispatch key. Exact three-key equality is therefore an unqualified author assumption, retained diagnostically rather than

imposed on original acceptance. This is a prospective contract correction before new model dispatch; neither the original criterion nor the contrary observations are overwritten. It is not a proof that every possible repair faces incompatible requirements. The supplemental native call's missing enclosing timer also remains unknown.

F Additional benchmark qualification

An independently pinned SWE-bench Verified metadata projection [3, 8] supplies 500 candidate repairs from twelve repository names. Ten of eleven prospectively selected two-issue pairs have forward base-commit ancestry; one has reverse ancestry in PR-date order. A separate current GitHub identity audit checks all 192 candidate/original repository pairs, finds no shared repository or declared fork identities, and flags two shared-owner groups. This does not establish historical code independence, portfolio separation or absence from model training. A later audit reports test-specification problems in a selected hard subset and evidence of solution exposure in tested frontier models [9]. Its selected-subset rate is not a whole-dataset error estimate, and it does not measure contamination in our provider profile. Public examples exposed during this literature review are separately recorded before any prospective split. No new confirmatory family or complete natural trajectory is enrolled.

Requests 6028 has a separately named ARM environment preserving its public Python version and thirteen listed Python package versions. The unchanged native SWE-bench 5.0.2 parser/grader observes all 187 designated checks in both controls: unfixed source passes 185 and fails the two required failing checks; reference source passes all 187 and is the only resolved control. Both collected outputs also contain eight missing-fixture setup errors outside the designated set. The original script lacks a test-exit marker, so a zero container exit cannot establish acceptance. This is designated-check qualification, not an error-free full-suite result or equivalence to the published x86 environment. Protected reference/test content remains in curator custody; model-facing selection reads public metadata only. Earlier import and decoding failures are retained. Infrastructure qualification contributes no provider-token or repair success observation for either treatment.

Requests 5414 subsequently qualifies in a new named ARM profile using its public development dependencies and Docker's default bridge network. Both controls complete all 131 designated checks: unfixed source passes 130 and the reference passes all 131. The full parsed outputs have 288 versus 289 passes, respectively, with one expected failure each and no setup errors. The original offline port, a dependency-build failure and a plugin-startup failure remain retained; the successful profile is a development correction. This does not make the original failed port successful. A later candidate grading service version fails its four-control qualification: three controls behave as expected, but this node's reference is not accepted despite all designated checks completing. Four existing public network-timeout checks instead encounter a network-unreachable condition; the earlier standalone run used the same image, package versions and evaluation bytes. The unique cause and a reliable remedy are not established by this discrepancy. Eight service rejection controls pass. This retained discrepancy illustrates why standalone benchmark qualification alone does not qualify a new endpoint. A subsequently frozen service version passes one attempt at each of its four native controls and all ten rejection controls. It preserves exact objective, profile, cumulative patch and resulting source-tree bindings. Public network readiness probes bracket Requests 5414's evaluation, with no automatic retries; an incomplete grade remains unavailable for repair decisions. All eight qualification probes pass. The 94-check audit retains 1,175 artifacts; the four control containers take 87.429894 seconds in total. This qualifies the named service and its observed controls, without proving the unique cause or remedy of the previous network failure or stability throughout future test runs. Candidate and benchmark tests still share the original test process; adversarial oracle integrity is not established. Requests 6028 retains its eight outside-set fixture errors.

An additive public-only cumulative-source control changes two production files and adds one Git-ignored Python file. The service inventories all 98 resulting files, matches the independently expected tree, and returns the unchanged unfixed grade of 185/187 in 6.048 seconds. This checks complete source-carry transport into grading, not successful repair or an accepted multi-issue trajectory. Earlier four-control cohorts remain unchanged.

A further pinned metadata-only projection yields 21,336 SWE-rebench rows [6] from 3,468 repository names and 731 SWE-bench Pro rows [11] from eleven names. The 22,067-row sum is not deduplicated

across datasets; protected issue, reference and test fields are not decoded in this projection. A prospectively seeded screen uses 2025 metadata dates, available image fields, distinct base commits and exclusion of prior repository/owner names. It yields 53 candidate pairs. All first sixteen pairs have actual forward Git ancestry, and 448 comparisons against the 28 prior current repository identities find no repository, declared-fork or owner match. Each candidate has a distinct current repository and owner identity. These checks do not prove historical code independence, portfolio separation or training novelty. Image-field eligibility is a convenience filter; all 32 selected image names specify x86_64, whereas the evaluation host is ARM. Source/environment/scorer qualification and a prospective confirmatory split remain open; zero new families are enrolled.

The first frozen additional pair, Zarr 2996 and 3015, subsequently passes four original native grading controls on explicitly ported ARM environments. Unfixed source passes 16/22 and 157/158 designated checks, respectively; each reference passes its complete set. Every control runs once, with zero retries or provider calls. The native test-spec factory, test commands, parser and designated criteria remain unchanged; the script port removes four Conda activation commands. The environments preserve Python 3.11.11/3.11.14 and all 50/67 observed installed distribution versions, while architecture, platform binaries and system-library builds differ. The local native fork reports version 4.0.3. Its complete declared dependency installation failed because an ARM YSON binary distribution was unavailable; a separately named local Docker backend uses the available Python implementation, leaving cloud backends unsupported.

The four native grader container intervals total 10.647 seconds, excluding environment acquisition, builds, Git preparation and audit. Earlier missing build-helper, project-version and Git-image-reference failures remain retained. The full public collection probe collected 3,045 tests for 2996 and exited zero; 3015 collected 3,123 but exited one during teardown with resource/unraisable warnings. The targeted controls do not reproduce that teardown failure, and their success does not erase it. As with Requests, the original script lacks a separate test-exit marker, and native shared-process execution does not establish adversarial oracle integrity. These results qualify the four original controls; candidate endpoint, complete trajectory and token economics require separate evaluation. No confirmatory family is enrolled by these controls.

A subsequent Zarr candidate endpoint qualifies all nineteen frozen controls, with 190 audit checks: six actual native grading calls, ten rejections before dispatch, two wrong-source containers stopped before hidden grading, and one duplicate-ID rejection. Empty and cumulative-public candidates retain the unfixed grades; reference production projections pass 22/22 and 158/158. Each projection preserves the full original reference's production and base-public bytes while omitting one ancillary added document, then reruns the unchanged designated checks. This qualifies the projected candidates, not transport of the full original patches. Non-Python edits remain outside this endpoint's admitted scope.

Exact public base inventories contain 189/181 files, including the editable installation's generated version module. Cumulative controls make three production edits, including a Git-ignored addition, and retain all 190/182 resulting files. The initial endpoint attempt failed before hidden grading because of raw mode differences and its rejection of the ancillary document; a later preparation attempt caught a no-op Git apply in a nested checkout. Both failures remain retained. The qualified revision preserves image-exact modes and verifies actual edits in isolated Git repositories. Six grader containers total 14.426 seconds within 30.363 seconds of outer service calls, including the duplicate-ID control. Preparation, environment acquisition and audit are separate. There are zero retries or provider calls. Context, accepted-source trajectory carry, publication and model efficacy are not enrolled by this endpoint qualification; adversarial oracle integrity and non-Python transport remain unqualified.

The second frozen additional family, Xradar, then completes its four original native controls on explicit ARM ports. Issue267 has 11/13 unfixed passes and 13/13 reference passes; issue269 has 55/58 and 58/58. All designated cases complete, with one attempt per control and no provider calls. Full original reference patches, including ancillary edits, are retained. The unchanged native factory/parser/test commands govern both nodes. Public collection yields 354 and 356 cases with exit zero. Each public prefix matches 106 original Git blobs, and all 22 data fixtures selected from pristine public tests match their registry hashes, totaling 183,051,686 bytes; both images fetch them offline.

Both ports preserve Python3.10.17 and all 196 installed distribution records, including two project distributions with natural imports resolving to the base-specific editable source. Four exact package versions require ARM source wheels. Conda binary/build variants and optional compiled-feature

parity remain unqualified. Retained failures include initial duplicate-project metadata ambiguity, a missing ARM binary wheel, and a node-orchestrator key error after the shared runtime built. Versioned corrections preserve runtime pins and do not retry grading. Failed and successful shared builds take 118.916 and 188.913 seconds; cached-wheel materialization takes 19.143 seconds, and the complete node-build/import/cache/package/collection phase 31.950 seconds. Four native grader containers total 69.044 seconds; these are scoped intervals, not complete recruitment cost. Environment and native audits pass 373 and 129 checks, respectively, with 120 custody artifacts and 59 native source files. Candidate endpoint, context, model trajectory, confirmatory enrollment and adversarial oracle integrity remain unqualified.

For a later development experiment on the existing pyparsing snapshot, the current public issue report supplies a concrete objective. Its nine examples reproduce the reported recursive-whitespace defect on unchanged public source: eight conform to the reported expectation and one fails. The report is frozen for that new experiment; its exact bytes at the original base date remain unverified. A reproduced public example is not complete hidden acceptance.

G Repository preparation and source custody

Repository context custody and preparation cost. A separate native producer/consumer qualification retains all 302 regular files across the XML and pyparsing development snapshots, including unparsed source. Its bounded semantic scope analyzes one of six and five of 154 Python files, respectively; four dependency capsules grant no source omission. Four cold/incremental root-and-block comparisons agree, four exact source expansions execute, and 22 negative controls reject. An added caller checks required-source CIDs and current bytes because native freshness omits that mapping; native "sufficient" flags are not behavioral authority. Full pyparsing scanning produces 3,957 symbols and 55,859 edges, but the unmodified producer process does not finish within its 600-second bound. Hoisting a repeated immutable node-CID index preserves the full XML state/root and reduces its preparation from 64.838 to 26.296 seconds in one observation per profile. Full pyparsing still reaches the 600-second bound after that change. Source retention and bounded integration therefore do not establish affordable whole-repository preparation, context compression or repair efficacy. Three later isolated profiles add a fixed-profile CID encoder, cached syntax validation and an exact literal-union predicate. Across all five profiles XML state/root identity is unchanged and every native consumer passes; each full pyparsing producer still hits the process deadline. In the last profile the producer writes its result at 597.222 seconds before the 600.202-second termination; the native consumer does not run. An earlier stage diagnostic had also assembled and verified a full bundle before timing out in source-binding reads. Missing final output therefore does not prove that bundle construction itself failed.

The literal-union change preserves all 55,859 actual link predicates against 3,957 node CIDs, an embedded-CID rejection, 1,235 constructed comparisons and 64 native regressions. The independently validated full bundle has 67,872 blocks and agrees with the last producer's state/root. Its 72.358 versus 2.834 seconds measure only one diagnostic predicate pass per implementation on a shared host; they are not a whole-pipeline speedup estimate.

A generic current-repository service separately checks exact source inventory, file modes and objective/request identities. Twelve controls pass across three XML preparations: a source change requires new state/root/pack identities, whereas an objective-only change retains state/root and creates a new pack. Both public Requests nodes then complete actual native producer/consumer roundtrips, retaining 97 files each and scanning all 35 declared Python files. Eleven negative controls pass per node; neither node permits any raw-source omission. These are context and custody checks, with zero provider calls and no repair-quality or compression comparison. Four subsequent Requests API expansions return 79,942 exact raw bytes across two real read processes, preserving source, request, freeze and objective identities. They add no omissions or model calls.

A sixth isolated profile memoizes identity only within an exact immutable native state instance under frozen codec/runtime bindings. Both full producer processes and their separate native consumers now complete. XML preparation takes 9.103 seconds; pyparsing takes 566.254 seconds, with 553.516 CPU seconds. The latter producer process takes 569.609 seconds and its consumer 1.740 seconds. All 302 regular files and 11,781,935 bytes are materialized; pyparsing's additional opaque symlink is retained without following it. Both sources' previous available state/root identities agree, including

the independently checked 67,872-block pyparsing bundle. All 22 binding/tampering controls pass and all 7,255 frozen runtime source files remain unchanged. Zero source omissions and zero signed receipts are retained, along with the unresolved semantic frontier.

The profile passes 159 native regressions. Separate full XML unchanged, comment-change cold and comment-change incremental states preserve serialized identities and all 1,992 reachable blocks across the parent and memoized profiles. Each profile performs twenty actual source materializations and rejects twenty corrupt-source reads. This qualifies immutable-object identity caching, not execution/proof reuse or a defense against reflective mutation of frozen objects. The successful full run establishes completion within the bound, not a replicated speedup or token benefit; the five previous deadlines remain results.

A subsequent frozen full-pyparsing comment-change comparison fails cold versus incremental equality. Its three workers complete within a separately declared 900-second bound: baseline 656.008 seconds, changed cold 609.801 seconds and changed incremental 784.021 seconds. Baseline and changed-cold workers run concurrently on the shared host; incremental restoration, full source guards, native previous-state/bundle loading, persistence and process exit are included in its worker interval. Earlier driver enrollment copy/freeze creation is not separately timed. These observations establish no incremental speedup.

The changed snapshot, symbols and artifacts agree, but 1,568 of 55,859 edge records differ: 1,564 ambiguous candidate edges change resolution metadata from `finite_may` to `definite`, and four derived references change with their edge identities. All retain conservative confidence and multiple candidates. Consequently state and semantic-root identities differ. Independent comparison reads every persisted block: 65,856 shared CIDs have identical bytes, with 2,016 distinct CIDs on each side, covering links, capsules and their indexes and roots. Both stores contain 67,872 blocks; their payloads total 112,728,034 cold versus 112,724,906 incremental bytes. Ten source-read controls pass and 7,255 runtime files remain unchanged. The driver exits one at the equality assertion after all workers exit zero; this is a retained graph-metadata failure, not another preparation timeout. A later actual native explanation-consumer probe loses the ambiguity limitation on all 1,564 directly changed edges while retaining their conservative-confidence limitation; four derived references are excluded. The fresh process takes 3.070 seconds and rechecks 7,255 runtime files. This establishes an explanation-output consequence, not an execution authorization or soundness failure.

A subsequent isolated guard refuses previous-state frontend reuse when graph projection has already replaced a lexical target with a concrete symbol. Such paths are reanalyzed from current source before whole-inventory resolution; the guard does not simply preserve stale resolution metadata. Seven changing inventory controls preserve state, roots, all block bytes and explanations. The full pyparsing comment-change comparison then restores exact serialized state/root equality and all 67,872 block payloads (112,728,034 bytes), also matching the earlier cold output. It reuses 20 of 154 Python paths, 172 symbols and 1,428 original frontend edges, while rescanning 134 paths. These counts are not omitted source bytes or avoided execution checks.

The two fresh workers run concurrently under unchanged 900-second bounds: cold takes 437.699 seconds and guarded incremental 614.522 seconds. The latter includes 107.173 seconds of prior-state restore/reverification and 55.736 seconds of scanning; graph construction and full verification still execute. The earlier 656.008-second baseline preparation is reused as an artifact and remains a separately incurred cost. Enrollment copying/freezing is not timed; the subsequent independent byte audit takes 3.625 seconds. Nine new source controls pass and 7,255 runtime file digests remain unchanged. Native regression tests retain 187 passes and one failure: an existing performance expectation requires reuse of a path that this correctness guard deliberately rescans. An earlier invalid authored fixture is also retained. A further inventory control exposes the guard's limit: adding a previously missing imported module leaves incremental confidence conservative while cold analysis becomes exact. The guard's successful comment-only comparison remains valid, but it is not a general lossless solution. No execution-soundness consequence is inferred.

A new profile instead persists the original Python frontend observations before pytest unification and graph projection. It binds their exact sidecar bytes to the accepted parent state, source inventory, namespace and extractor/runtime profile; unsupported or stale inputs fall back to current analysis. Current pytest analysis and whole-inventory graph resolution still run. The full comment-change comparison reuses 153 of 154 Python paths, 3,885 original symbols and 52,977 original edges, extracting 772 edges from the changed file. Cold and incremental serialized state, root, all 67,872

blocks/112,728,034 bytes and the complete sidecar agree. The native state and blocks also equal the earlier cold output. Each sidecar adds 53,753,923 bytes of storage. Ten bounded equality cases and sixteen rejection/fallback checks include the earlier unresolved-import confidence case; ten full-worker source controls pass and 7,256 runtime hashes remain unchanged. Accepted honest producer custody remains an assumption: hash binding alone does not prove that arbitrary observations follow from source.

All three new workers finish within frozen 900-second bounds. Baseline sidecar bootstrap takes 231.354 seconds in addition to the separately paid original 656.008-second preparation. Changed cold takes 440.276 seconds and incremental 607.971 seconds. Incremental includes 105.262 seconds of parent restoration and verification, 55.520 seconds of scanning with observation decode/encode, 390.254 seconds of bundle construction, 48.819 seconds of full verification and 4.727 seconds of persistence. Bootstrap overlaps changed cold; incremental follows bootstrap, for 839.328 seconds of new worker schedule time. Enrollment takes a separate 0.533 seconds and the independent audit 5.958 seconds. Nested stages are not added again to process totals. This qualifies actual original-observation reuse on one full public change; incremental preparation is still slower, and zero source omissions or provider calls establish no token saving or general speedup.

H Combined controller, publication and recovery

First combined controller qualification. A new caller joins actual native context reads, source-bound semantic analysis, symbolic routing, compiler-mediated world memory, isolated behavioral scoring and a durable effect journal. On the four existing constructed routing tasks, two supported initial repairs pass and two remain residual-pending with the provider disabled. Authored follow-up events consume accepted source, recheck a comment change and reject a stale local parent. Ordinary no-op detection receives the same current behavioral checks as the memory path. Three actual process interruptions exercise validated continuation, durable receipt reconciliation and an executed effect whose completion remains unknown; the unknown effect is held without redispatch. Across thirteen events there are nine local acceptances, two pending residuals, one stale parent and one unknown effect. These event counts are not thirteen independent repair tasks. Thirty scorer attempts yield 29 durable terminal receipts. Local ledger CAS is not native Kit publication; sealing, lifecycle reuse and certified procedures are not yet combined here. With zero provider calls this qualification establishes no integrated token-savings effect.

Subsequent native publication profile. The separately frozen eight-event sequence yields six acceptances, one stale branch and one pending residual. A full checkpoint at generation 0 is followed by a delta at generation 1. A real service exits with code 94 after native seal persistence and before CAS. A fresh process loads the pending seal, rechecks the entire parent chain and exact local evidence, replays visible and audit behavior, then completes native CAS/WAL through an explicitly added development adapter, reaching generation 2. Another fresh event consumes the native stored accepted source and publishes generation 3. A branch based on generation 0 reaches native publication and loses with a generation mismatch.

Table 4: Actual work in the eight-event native publication qualification. Event outcomes are dependent continuation observations.

Operation	Count
Controller invocations / terminal effects	9 / 68
Native context reads / source expansions	4 / 8
Input scans / accepted-candidate scans	8 / 3
Symbolic attempts / produced candidates	4 / 3
Native routing calls	8
World-memory service calls / compiled actions	11 / 2
Publication attempts / native reads / recovery	7 / 4 / 1
Controller / additional admission scorer processes	18 / 24
Provider requests / provider tokens	0 / 0

Invocation wall time is 60.906438 seconds; the additional 24 behavioral replays consume 13.759197 seconds within that total, not as a separate additive cost. Context enrollment, inventory freezing,

implementation and read-only audit are outside this runtime denominator. Crash-time work is retained through a durable receipt, and recovery does not double-count its aliased completion. Both ordinary and memory paths run the same cheap current-source check and accept no-op revisits without symbolic or model repair. The compiled memory action dispatches an actual check; no reduction in checks is demonstrated.

The complete required leaf set is externally frozen: accepted source, semantic preparation, effect inventory, visible behavior and audit behavior. Every changed candidate receives a new native scan. All five leaves are replaced per delta. Native integrity verification checks content and digests; unsigned local receipts additionally depend on trusted controller/scorer execution, a durable effect ledger and an evaluation-owned CID index. This is neither a theorem about arbitrary Python nor production trust admission. Stored parent chains are recursively checked and the requested source is behaviorally replayed on read. Broader crash phases, concurrent publication, provider effects, real repository edits and lifecycle reuse remain open.

The read-only audit passes 6,424 checks. The archive retains 504 source and metadata files, including 483 native source files across the loaded closure and preceding context enrollment. The first profile's omitted imported wrapper and the first audit's pointer-envelope parser failure are retained. Correcting the audit did not rerun behavioral outcomes; the archived previous controller still passes its original 9,188 checks.

Full-repository service fixture and measured cost. A separate curator-held Requests 6028 fixture begins with its reference candidate and then adds a neutral comment to the same accepted production source. Both states contain all 97 source files. It executes two full native context preparations, four world-memory service calls and six original native grader invocations: two initial checks and four fresh publication, interruption, recovery and read replays. All six complete the 187 designated checks. Reference and hidden test payloads remain in curator custody. This fixture does not execute the Requests 5414-to-6028 upstream transition or any model repair, and it does not establish adversarial oracle integrity.

The first full checkpoint reaches generation 0. The second process constructs a delta and actually exits with code 94 after seal persistence but before CAS. A fresh process recovers generation 1; another fresh process reads and restores the exact second accepted source. Wrong-tree, wrong-receipt, source-drift and incomplete-effect controls all reject before grader replay. All four required leaves—accepted source, semantic preparation, observed non-sealing effects and native behavior—are replaced on transition. The effect leaf excludes all sealing operations, including prior reads; the outer workflow ledger must still count their actual work. Parent/source admission and fresh grading are additional checks. Unsigned local receipts and native integrity mode establish neither a theorem of execution nor verification reuse.

Table 5: Measured disjoint sequential intervals for the Requests 6028 curator fixture. Grader processes are nested in these phases and are not added again.

Phase	Count	Wall seconds
Full native context preparation	2	318.283
Initial accepted-candidate checks	2	16.047
World-memory service calls	4	5.858
Publication/recovery/read and rejection controls	8	37.037
Sum of measured intervals		377.226

The six grader containers consume 31.625 seconds within the measured intervals. Their four replay service wrappers are also nested, not independent extra costs. The frozen-start-to-result artifact span is 377.640 seconds; this is not a separately instrumented whole-process timer and excludes earlier imports and profile creation. Later code archival and read-only audit take 0.275 seconds together, outside the fixture intervals. There is one shared-host observation, no parallel comparison and no provider calls. The 275-check audit retains 33 public code snapshots and hashes 1,136 private artifacts. This closes the stated service controls while leaving multi-issue model efficacy, token savings and broader crash coverage open.

Natural Requests sequence: intervention and failure paths. The first live repository cohort contains one development family, two genuine public issue nodes and four arms, executed in frozen D/C/A/B order with independent local cold stores. Each trajectory has at most four provider requests, with at most two at its first issue; unused slots carry forward. All planning, proposal, expansion and recovery purposes share that ceiling. The common Grok 4.6 low-reasoning profile has a 2,048-output-token limit and a 65,536-byte request limit. Source retrieval selects whole production modules with three-file/48,000-raw-byte limits, and exact requested expansions replace the earlier selection. Accepted first-node source is committed on its exact public base and carried through a real three-way merge with the next base. No reference patch enters the model or its public history.

A uses ordinary lexical retrieval and the common current-source check. B actually prepares and reads native context, but zero omissions qualify, so its model context falls back to the same raw source. C additionally runs native analysis, bounded synthesis and routing; its native candidate fails before the model repairs the first issue. D adds actual world-memory services, accepted-source context preparation, native publication and stored-source read. It remains partial: lifecycle reuse, certified procedures, learned world utility and integrated theorem proof are absent. Native permitted model routes share the same fixed provider profile, so model-tier selection cannot explain cost differences. Each arm retains its own histories and state; the effect journal provides durable identities for every arm.

All four initial requests have identical canonical bytes and 13,693 provider input tokens. Each returns one independently passing first-node repair. A and B receive 13,568 cached input tokens each, versus 512 each for C/D; local cold state therefore does not imply equal remote cache state. Returned reasoning and output lengths vary. Small first-node total-token differences cannot be attributed to source compression or a witnessed avoided model call.

A/C/D's accepted edits each conflict in the next public source merge. Their retriever appends the required conflict module to its ordinary selection, yielding four modules, 80,552 raw source bytes and a 91,202-byte request. The frozen 65,536-byte gate halts before dispatch. B's different initial edit merges without a conflict and its next request fits at 53,319 bytes. Its three subsequent candidate grades are 184/187, 185/187 and 185/187, all unsuccessful; the original current source passed 185/187. The second node thus fails in every arm, through different observed paths. Source-bound halts are retained failures, not successful symbolic closures.

The complete fixed-cohort provider totals are 14,428 (A), 65,695 (B), 14,589 (C) and 14,269 (D), summing to 108,981 across seven actual posts. Each arm has one accepted node, so tokens per accepted node equal those fixed-cohort totals; tokens per completed trajectory are undefined. All unsuccessful model attempts remain charged. The first-horizon totals are 14,428/14,192/14,589/14,269 in A/B/C/D order. These are observed cumulative costs, not counterfactual quality-versus-budget curves or independent repeats. Charges, cached input, visible output and reasoning components remain separate in the retained provider receipts; account settlement is not inferred.

The original driver terminates with exit 143 while B's first controller is still running. That same worker finishes and commits its accepted event. A separately frozen reconciliation checks the exact terminal output, journal, source/profile identities and absence of pending effects, then executes only the original remaining B6028 event. B5414 is never restarted. Its outer invocation time, waited CPU and exit status remain unavailable; the controller's own 209.770-second timer survives. B's two controller intervals sum to 408.510 seconds, which is a different timing scope from other arms' enclosing invocation intervals. Docker workload CPU is unavailable throughout. Preparation, grading, memory and publication/read phases are nested within runtime and must not be added twice. Development qualification and source/environment acquisition remain separately scoped overhead. A/C/D enclosing intervals are 96.745, 285.790 and 507.358 seconds, respectively; B's enclosing total is unavailable.

The separate ledger audit passes 1,769 source, journal, request, attribution and budget checks. It joins all seven model receipts to their actual effects and dispatch intents, with no unknown provider effects. Sixteen unique native grader receipts include the two additional D publication/read replays; repeated appearances of saved completions are not new calls. Model and effect accounting are complete for this cohort, while outer runtime accounting remains incomplete because B5414's enclosing timer is missing. The source archive contains 504 files. Preparation records include observed native imports; the source-expansion process lacks a separate actual import receipt, which limits the demonstrated execution closure. The quantitative result is not a production trust admission.

One family with implementation failures and uncontrolled server caching cannot identify a general mechanism effect. Equal accepted-node counts also conceal the different second-node opportunities. No arm completes the intended natural trajectory, and the broader cache, warm-horizon, ablation and independent-family program remains required. Subsequent transport corrections receive a new profile; these outcomes and request bounds remain unchanged.

Second natural Requests cohort. A separately frozen conventional retrieval correction gives mandatory conflict modules priority, followed by optional lexical source that fits the existing three-file/48,000-raw-byte/65,536-request-byte bounds. A detached driver retains all eight outer invocation timers without restart. On actual B/D continuations, the conflict and two optional modules fit in 47,424 raw bytes and a 54,773-byte request. A/C's different accepted first edits merge cleanly. All four first requests remain byte-identical within this cohort. No source omission or successful symbolic closure occurs; D still lacks integrated lifecycle reuse, certified procedures and learned memory utility.

All sixteen calls execute and consume 243,255 provider tokens, including expansion decisions, failed proposals and local transport failures. A uses two calls per issue, while B/C/D use one then three. Each accepts only issue5414. A's first proposal passes 123/131 designated checks; the next passes. For issue6028, A spends one call expanding source and one on a candidate grading 185/187. C expands once and grades two candidates at 185/187. D grades its first and third candidates at 185/187; their cumulative patch bytes are identical. B grades one candidate at 185/187. The remaining three outputs, one D and two B, restore all 97 pristine file contents but retain mode0644 instead of0664 on one file. The frozen transport rejects these mode-only differences; its text-edit interface has no mode channel. Those three provider calls remain charged, no graders run for them, and their counterfactual grades remain unknown. No retrospective repair or rerun alters this cohort.

Fixed-cohort tokens are 56,543/62,439/63,336/60,937 in A/B/C/D order; first-node cumulative tokens are 29,787/14,782/14,364/14,163. Tokens per completed trajectory are undefined. Total components are 227,336 input, 1,804 visible output and 14,115 reasoning tokens; 47,744 cached input tokens are a subset. Cached-input counts are 1,024/30,976/14,720/1,024, so local cold starts do not control provider caching. API-reported charges total $0.1627138000; settlement is unmeasured.

All 21 unique grader calls complete: nineteen direct and two D publication/read replays. Six passing and fifteen failing grades include repeated validation of the same task and are not independent success counts. Their 509.499 seconds of container time are nested within 544.910 seconds of outer grader service, which is itself part of runtime. The 53 unique outer effects have complete elapsed timers. Enclosing invocations total 1,382.526 seconds. Native context contributes 106.468/211.192/213.978 seconds for B/C/D; D publication/read, including replays, takes 78.228 seconds. These components are not added twice. Docker CPU and driver-preflight CPU remain unknown. The logical runtime-state snapshot contains 5,495 files/297,870,884 bytes, excluding qualification and private scorer stores; it is not deduplicated physical usage.

The prospectively fixed detailed audit passes 3,698 checks; independent root aggregate and final joins reconcile all costs, nodes and 510 archived sources. The final root join passes 1,412 checks after retaining an initial projection-ID assumption failure and joining explicit residual/enclosing event identities. The curator independently joins all 21 requests, dispatches, receipts and outer completions, with no unknown effects. All 310 observed worker/world/sealing imports match the archive; a separate native context audit binds 163 observed producer/consumer paths and all ten context effects. Expansion lacks a separate actual-import receipt, and standard-library/third-party import closure is not established by these native-prefix receipts. The follow-up supports no token saving and is not pooled as an independent confirmatory replication. Original cache, operational-sequence and component comparisons remain required.

I Native proof profiles and exact statement checks

A real native proof relation. The copied native backend was built offline with locked dependencies because the bundled binary did not implement the source's v3 circuit. Isolated development keys support a four-leaf Event-DAG Merkle/count relation. Three actual proofs for counts 1, 2 and 4 verify; eight witness, public-input, proof and key mutations reject. Primary setup takes 7.810 seconds, proof generation 12.048–12.474 seconds, and verification 3.053–5.147 milliseconds; serialized proof JSON

is 1,484 bytes. The raw verifier accepts a different outer root that maps to the same field element. The evaluation caller's exact-root/count and key/binary checks reject that alternate statement before dispatch. This boundary is part of the result: the native relation alone does not bind arbitrary outer metadata. Development setup is not production trust admission, and a Merkle relation is not proof that arbitrary tests executed. The v5 pytest-attestation circuit remains unsupported in this backend.

The original bounded Horn profile. The source-built v2 backend also produces and verifies three honest implication chains of lengths 1, 2 and 4. Setup takes 1.101 seconds, proving 2.238–2.390 seconds, and verification 3.187–5.267 milliseconds; each proof JSON is 1,700 bytes. The honest prover rejects the tested incomplete or inconsistent inputs. When the scalar public inputs remain unchanged, the raw verifier accepts three changed outer metadata fields; exact statement and source/key checks at the evaluation caller reject them before dispatch. Honest examples and wrapper checks do not establish that every satisfying circuit witness corresponds to the advertised textual axioms. That stronger assurance remains unconfirmed.

Actual accepted-history archive integration. A separately frozen producer/consumer profile follows the constructed controller's native generation-3 pointer through four accepted seals, each with five required receipt units. It excludes the stale unaccepted branch and the separate ordinary trajectory. Producer and fresh consumer use native Kit verified reads on exact archive clones, reconstruct the original local effect receipts and check the complete source, parent and leaf bindings. The original archive and controller ledger remain unchanged. No behavioral checker is rerun; the original local execution remains the authority for its recorded observations.

Each canonical event commits its seal/parent, source and original binding, complete receipt-CID map and verified pack digest. The already qualified standard v3 backend proves their four-leaf archive relation. The consumer checks the externally expected full root/count and exact binary/key, in addition to the accepted pointer and chain. The raw scalar-root limitation still applies; the proof does not establish that arbitrary code or tests executed correctly. The stopped custom-v2 investigation is not used.

One honest proof and one runtime pairing verification complete. Producer and fresh-consumer read processes take 0.325531 and 0.342170 seconds, proving 12.617511 seconds and verification 0.006725 seconds. The measured driver totals 14.749495 seconds, including source/key/archive hashing, clones and negative controls. These phases are nested. Earlier backend compilation, the original 7.810198-second development key setup and historical controller execution are separately scoped costs. A later audit adds a 0.006085-second native verification. This is one shared-host observation with existing keys, not a replicated speed comparison or evidence that proving beats direct hashing.

All seven frozen controls reject: missing/corrupt receipt bytes, incomplete leaf inventory, stale parent, reordered events, changed event source commitment and changed full root. Four exercise the native reader; the last three reject at exact statement binding before pairing. The proof JSON is 1,484 bytes; the original 69-file store contains 2,010,722 bytes, including additional unselected records. A small proof does not replace the original source/receipt custody that the consumer still reads. The separate audit passes 218 checks, recomputes all event digests and the full Merkle root, and archives 34 observed source files. The audit shares the cryptographic backend. This establishes no new behavioral check, model call, execution reuse, token saving or production admission.

Actual Requests accepted-source archive. A separate post-cohort consumer uses the actual Requests cold-v2 D5414 checkpoint. D6028 failed and contributes no accepted seal. This single event has four required units: accepted source, native behavior, semantic preparation and complete non-sealing effects. Two fresh processes read exact archive clones through native verified-byte storage and admission, recheck original source, patch, objective, environment, context and effect bindings, and restore all 97 files with exact bytes and raw modes. The original 118-file archive and ledger remain unchanged. This explicit archive load dispatches no new grader; the historical controller's ordinary read included a grader replay.

The event digest includes the complete pack, required receipt CIDs and source inventory. One honest standard v3 proof commits its root/count with three padded leaves. Proving takes 11.978556 seconds and runtime verification 5.255706 milliseconds; direct producer/consumer read-and-restore take 0.419219 and 0.423702 seconds. The complete 14.737743-second driver includes these costs, cloning,

hashing and all controls. An additional 0.278576-second audit includes one 7.364014-millisecond verification; previously metered key setup and cohort execution are separate. The proof is 1,484 bytes, while its supporting archive occupies 12,514,534 apparent bytes. This consumer retains direct evidence checks; the proof adds overhead and establishes no amortized or token saving.

Eight frozen mutations reject. Missing/corrupt behavior receipts, omitted units, changed parent and changed source binding fail archive admission. Changed event, full root and count fail exact outer-statement binding before pairing. The changed-parent checkpoint is looked up under delta kind and rejected before cycle traversal; the initial auditor expected a later kind/cycle error. Its failure is retained, and a separate corrected auditor checks that exact earlier lookup rejection without rerunning the experiment. The audit passes 1,462 checks and archives 514 source files. It shares the cryptographic backend. This is actual repository archive consumption, not inline proof use in the original cohort, a proof of arbitrary test execution or production admission.

J Signed native procedure execution

The surviving saved bounded-sequence candidate now passes native seven-layer verification, certificate issuance, certificate admission and TEST interpreter execution. Its exact procedure identity is retained. A local TEST-only HMAC signer and trusted immutable-input custody provide this development admission; the selection receipt grants no native registry promotion or production authority. The arithmetic candidate is not relabelled as a repository-repair procedure.

The prospectively frozen corpus contains 32 test, 24 adversarial, 48 held-out and 32 shadow inputs, disjoint from the original exposed inputs and one another. All 136 native executions match independent decisions: 13 true and 123 false. False is a computed answer, not an execution failure. Six handlers per input produce 816 dispatches, 408 distinct current check receipts and 136 distinct decision receipts. The audit reconstructs exact invocation CIDs and restores all native trace, outcome and receipt objects. These newly authored partitions were prepared after development; they are not population-generalization evidence.

Eight distinct proof, test, adversarial, held-out, shadow, specification, source-episode and counterexample artifacts precede signing. Seven bounded Z3 contracts are connected to 4,100 concrete catalog executions; this is a bounded logical result with a measured source bridge, not a theorem for arbitrary Python. The original IR has an empty required-proof-contract list. An added adapter enforces the named proof payload; native contract-specific proof coverage is not inferred. Independent artifact admission recomputes concrete decisions, and runtime validation repeats catalog operations. These costs are charged; this experiment does not demonstrate receipt-only computation avoidance.

Thirty pre-issuer evidence controls and thirteen native runtime controls pass, including signature, expiry, catalog, observation, query identity, cross-invocation and domain mutations. LIVE dispatch rejects. Exact invocation checkpoint reuse avoids redispatch in the same interpreter process; cross-process checkpoint restoration is untested. Since native canonical values reject floats, a frozen canonical-JSON transport preserves input bytes behind a checked typed query CID. The evidence corpus retains 1,192 float-bearing adversaries.

Preparation and freeze take 1.402121 seconds. Three fresh bounded workers take 1.521004 seconds for evidence production, 6.902933 seconds for admission and issuance, and 8.819003 seconds for execution and controls: 17.242940 seconds total. The 6.771036 seconds of native invocation calls and 5.127014 seconds of evidence-negative work are nested. Independent audit adds 2.929698 seconds. Evidence payloads occupy 2,373,459 bytes and native runtime objects 2,704,141 bytes, excluding snapshots and logs. Earlier float, immutable-mapping and SMT serialization failures retain their original profiles, limits and costs. Profile copying before the driver was not timed. The freeze pins 10,736 paths; 156 observed loaded files include 112 standard-library/other paths outside the native source freeze. Local custody does not supply OS isolation against hostile Python. No provider call, model-token saving, repository transfer or production authority is attributed to this integration.

K Family-level display

Figure 1 displays the same family counts as Table 3; it is not an additional experiment.

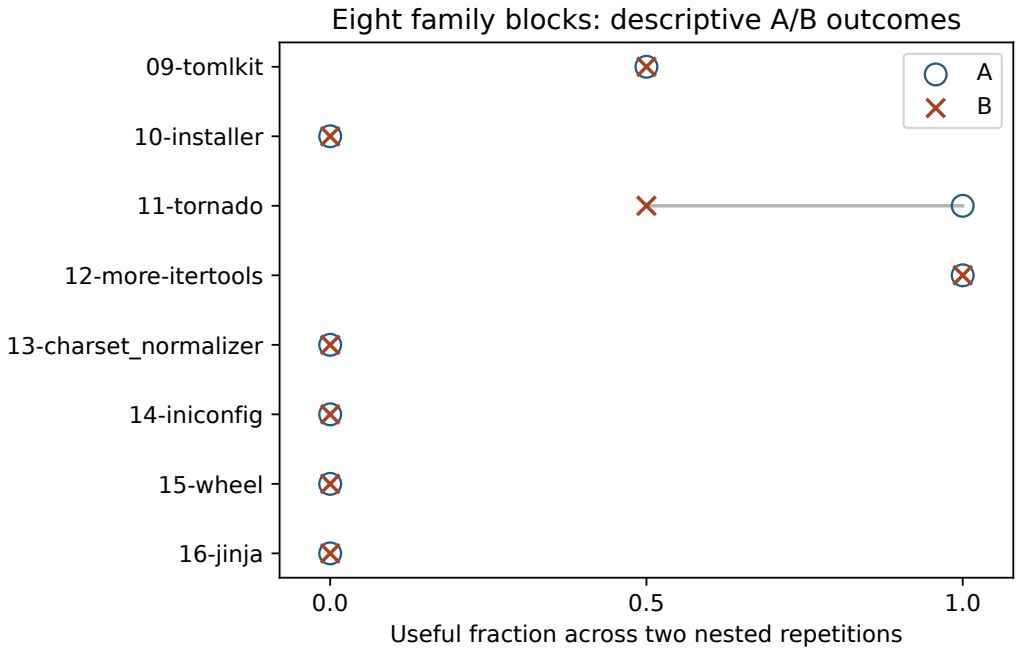


Figure 1: Useful completions by family and arm in the same 32-cell comparison. Each arm has two nested repetitions per family; the figure adds no trials.

L Historical boundary evidence

Table 6 retains the historical qualification scope separately from the final repair comparison. Identifiers of the form NS-NNN name internal qualification records, not repository families or independent repair trials. The anonymous boundary component retains the detailed evidence index and limitations.

Table 6: Historical evidence boundaries. Local controls, unavailable measurements, and the retained false-reuse case are not additional final repair outcomes.

Boundary	Safe rejection	Valid progress	Evidence
Provider gate	Missing/mismatched authority and untrusted close rejected in fixtures.	Deterministic repair and injected residual fixture; no production routing claim.	NS-008
Loss-aware translation	Modal erasure and unvalidated solver lift refused.	Exact/conservative translation only; no kernel source proof.	NS-009
Native theorem/circuit	Untested: no admitted native proof profile.	Unavailable: no genuine proof or measured prove/verify run.	NS-014 scope closure
Fixture/call/teardown reuse	Prior false reuse 1/26 retained; corrected route requires full cold.	Full public cold baseline only; no admitted positive reuse or speedup.	NS-011; corrected NS-027
Publication after restart	Empty/stale/failed publication refused in local controls.	WAL/CAS recovery and valid continuation; explicit hermetic shims.	NS-012/013
World/procedure consumption	Untested/out of scope.	Untested: no retained paired pre/post consumption.	NS-015 scope closure

M Withdrawn and unmeasured comparisons

Table 7: Withdrawn and unmeasured comparisons. These entries are scope dispositions, not zero-effect measurements.

Endpoint	Scope
C/D routing/governance comparative benefits	Withdrawn or unmeasured; no final comparison or effect estimate.
proof/test reuse or local_warm effects	Withdrawn or unmeasured; no final comparison or effect estimate.
certificate/publication comparative benefits	Withdrawn or unmeasured; no final comparison or effect estimate.
adversarial scorer-integrity qualification	Withdrawn or unmeasured; no final comparison or effect estimate.
independent human semantic annotation	Withdrawn or unmeasured; no final comparison or effect estimate.
Non-A/B planned cells (128)	Withdrawn; no measured ablation rows.
A/B warm planned cells (32)	Withdrawn; no measured ablation rows.

N LLM use, public reproduction, and responsible use

Scientific model use. The historical cold-repair comparison uses Grok 4.6 to propose source edits from the fixed A/B context requests (Section B). Each of the 32 cells consumes one proposal POST with a maximum of 4,096 requested output tokens and a structured edit response. No scientific fallback, retry, served random seed, or reasoning-effort parameter is used. The model's weights and exact served revision are unavailable. Candidate inspection by AI is a procedural control; it is not independent human assessment or a substitute for executed checks.

The new routing and context development pilots each use six Grok 4.6 requests under a separately frozen low-effort profile, with at most 2,048 requested output tokens and 240 seconds per request. These pilots do not share the historical experiment's reasoning-effort setting or task population. All twelve responses retain actual model identity and token-usage fields; no scientific retry, model fallback or auxiliary model call is included in these development workflows. Deterministic analysis, solver checks and executed oracles remain distinct from model proposals.

Research and manuscript assistance. AI assistance also contributed to source recovery and drafting, implementation and debugging, experiment preparation and candidate inspection, and analysis and artifact preparation. The original authoring supervisors were configured to use Grok 4.6, with Codex `gpt-5.6-terra` at high reasoning effort as a fallback only following independently verified primary-quota exhaustion. This describes the configured authoring policy, not a claim that every task used a fallback or that these models generated the scientific comparison under a mixed route. The configured policy does not identify the model behind every separate interactive assistance session. AI-assisted checking supplies no measured human semantic fidelity, expert agreement, or human effort time, and does not discharge the conditional arguments in Appendix A. The present incremental evaluation code, reports and manuscript revision also received interactive Codex assistance. That authoring effort is not included in the measured deployment-task token totals; no full research-effort saving is claimed.

Anonymous numerical reproduction. The historical supplement keeps the 32-cell final comparison, the separate 24-cell developmental pilot, and historical boundary controls in distinct components. Its top-level `README.md` specifies Python 3.12 and the command `python3 -B reproduce.py -root ROOT -output OUTPUT`, where `ROOT` is a fresh extraction and `OUTPUT` is a fresh directory outside it. The program checks the manifest-bound files and reproduces their retained numerical summaries; it does not generate new proposals, execute candidates or hidden tests, rerun historical qualifications, or authenticate original signed records. Hidden tests, reference patches, credentials, keys, and raw private operational logs are excluded. Original and derivative hashes have distinct roles: original signatures do not authenticate modified anonymous derivative bytes. Exact executed source and labeled reference derivatives remain distinct. Upstream retrieval metadata, revisions, and retained license and copyright notices accompany the released assets. Stable hashes and timestamps may permit correlation, so these release controls do not guarantee anonymity. New development artifacts are retained separately with source and request hashes, terminal outcomes and replay commands. They have not been incorporated into that historical anonymous supplement; its reproduction command does not reproduce the new evaluations.

Potential impacts and limits. Explicit evidence boundaries may help users identify stale checks and unsupported acceptance claims, but this experiment establishes no general reliability or efficiency benefit. Incorrect repairs, incomplete tests, or mistaking signed provenance for correctness could lead to unsafe deployment; automated coding can also be misused to introduce harmful changes. The stated scope, isolated checks, restricted artifact projection, and visible failures reduce particular risks without establishing comprehensive adversarial resistance. No model weights or human-subject dataset are released, and no human semantic annotations, expert-agreement study, or human-time measurements were collected for the reported comparison.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and Sections 1, 5, and 8 restrict the contribution to explicit evidence boundaries, conditional arguments, and the fixed eight-family A/B comparison (A 5/16, B 4/16). They do not claim semantic correctness, a general performance advantage, or benefits from withdrawn routing, reuse, or publication comparisons.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 8 states the convenience-sample scope, combined-context intervention, finite-test and trusted-premise limitations, and missing human semantic fidelity, expert agreement, and human-time measurements. Sections C and 5 retain failures, timing deviations, overlapping clocks, incomplete validation, and unsettled charges.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [No]

   Justification: Section 3 and Appendix A give explicit scopes, premises, and conditional argument sketches, but do not claim a complete formal proof of the intended implementation. The arguments are unmachine-checked, and dependency completeness, checker adequacy, and exclusive checked publication remain assumptions.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [No]

   Justification: Sections B and C disclose the retained protocol and frozen reduction, and Section N describes the anonymous numerical replay. Independent regeneration and confidential-oracle rescoring are not supplied: exact served model revisions, private original custody, and hidden tests are unavailable through that replay.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [No]

   Justification: The anonymous supplement provides manifest-bound inputs, selected source versions, retained license/retrieval metadata, and Python 3.12 instructions that reproduce the reported scalar summaries. It does not provide model weights, hidden-oracle access, or public authentication of original private signatures; Section N states these limits.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Section B specifies the historical cold comparison, and Section 6 distinguishes the new constructed development profiles, calls, oracles, cost scopes and limitations. Frozen per-study artifacts retain the exact requests, execution settings, cases and source bindings; the development observations are not pooled with historical results.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [No]

Justification: The historical study retains its explicitly descriptive whole-family bootstrap interval, but the new small constructed development pilots do not provide population-level uncertainty estimates or statistical significance claims. Their observed savings are limited to the fixed cases; held-out repeated comparisons remain necessary.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [No]

Justification: Sections B, C, and 5 report per-phase CPU, memory, process and time limits, available measured clocks, prior work, and missingness. Provider-side hardware and total compute/effort or settled charges are unavailable, and overlapping clocks, reservation lifetimes, and configured caps are not presented as measured total compute.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [N/A]

Justification: Author review of and conformance with the NeurIPS Code of Ethics has not been attested in the preparation record. This answer records unavailable author confirmation, not a claim that the work violates the code; the author must review it before submission.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.

- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: Section N discusses potential benefits of making evidence boundaries explicit, and harms from incorrect repairs, misplaced trust in signed provenance, unsafe deployment, malicious edits, and residual privacy correlation. It states the limited scope of the proposed mitigations and makes no measured societal-benefit or comprehensive security claim.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The release contains no model weights or human-subject dataset of the high-risk types described by this question. Section N and the supplement documentation nevertheless describe excluded private logs, credentials, keys, hidden tests and reference patches, and the residual limits of anonymization.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.

- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [No]

    Justification: The supplement retains upstream revisions, retrieval metadata, license and copyright notices, and distinguishes exact executed source from reference derivatives. A complete confirmation of all provider and asset terms of use is unavailable in this preparation, so retained notices alone are not asserted to establish full compliance.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The supplement documents its final-comparison, developmental-pilot, and historical-boundary components with separate manifests, numerical reproduction commands, source and derivative identities, and explicit data-access and inference limitations. Section N explains this scope, including the distinction between integrity hashes and authentication of private originals.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

Justification: The reported source-code comparison includes no crowdsourcing or human-subject annotation study. Section 8 states that human semantic fidelity, expert agreement and human time were not measured; AI candidate inspection is not relabeled as human research.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No human-subject experiment or crowdsourcing is reported, so no participant-risk, compensation or IRB-approval result is claimed. The absence of human semantic annotations and expert-agreement measurements is explicit in Section 8; this answer does not invent institutional approval.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: Section B and Section N identify methodology-essential Grok 4.6 scientific proposals under one-POST/no-fallback rules and describe AI assistance in implementation, inspection, analysis and writing. The configured authoring Grok-to-Codex `gpt-5.6-terra` high quota-only fallback is distinguished from observed scientific use, without claiming that every fallback or interactive session used that model.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.